MealTrace Digital

Internal Development Plan

Project	MealTrace Digital – Tier 1 MVP
Document Date	April 2026
Version	v1.0
Backend Developer	Harshit (True Live Lona)
Frontend / App Developer	Rakshit
Client	Manish – Catering Operations

1. Overview & Team Responsibilities

This document defines the internal development execution plan for MealTrace Digital, describing how the build work is divided between Harshit (backend + Web UI) and Rakshit (UI/UX planning + Flutter Android app). It supplements the client-facing SOW and is intended for developer use only.

1.1 Role Split

Area	Harshit	Rakshit	Notes
Backend / APIs	✅ Owner	—	Python (FastAPI/Flask), Firebase
Database Design	✅ Owner	Review	Firebase Firestore + schema
Firebase Services	✅ Owner	—	Auth, FCM, Hosting, Storage
Payment Gateway	✅ Owner	—	Razorpay integration & webhooks
UI/UX Wireframes	Input	✅ Owner	All 3 roles: Resident, Vendor, Admin
Screen & CTA Planning	Input	✅ Owner	Flow diagrams, screen list
Web UI (PWA for iOS)	✅ Owner	Review	React / HTML+CSS, API integration
Flutter App (Android)	API Support	✅ Owner	Flutter, Play Store publish
Deployment & Hosting	✅ Owner	Play Store publish	Firebase Hosting + Vercel (PWA)

2. Phase-by-Phase Development Plan

The project is divided into 5 sequential phases over 14 working days. Phases 1–3 must complete before Phase 4 begins, as frontend integration depends on hosted, tested APIs.

Phase 1 — Backend: Database, APIs & Firebase Setup (Days 1–5)

Owner: Harshit | Dependency: Client provides resident CSV, site list, and meal windows by Day 1.

PHASE 1 · BACKEND BUILD · HARSHIT (Days 1–5)			
Day(s)	**Task**	**Owner**	**Details / Deliverables**
Day 1	**Firebase Project Setup**	**Harshit**	Create Firebase project; enable Firestore, Auth (OTP + email), FCM, Hosting, Storage. Define environment variables.
Day 1	**Database Schema Design**	**Harshit**	Design Firestore collections: residents, sites, scan_logs, plans, payments, guest_passes, admin_users. Share schema doc with Rakshit for UI planning.
Day 2	**Auth API**	**Harshit**	POST /auth/login (email+OTP), POST /auth/refresh-token, POST /auth/logout. Role-based middleware: RESIDENT, VENDOR, SUPER_ADMIN.
Day 2	**Resident APIs**	**Harshit**	GET /resident/profile, GET /resident/qr-code (UUID + signed payload), GET /resident/balance, GET /resident/transactions (paginated).
Day 3	**QR Code Generation**	**Harshit**	Generate cryptographically signed resident UUIDs. QR payload spec: resident_uuid + site_id binding. Expose GET /resident/qr endpoint returning base64 QR image.
Day 3	**Scanner / Scan Validation API**	**Harshit**	POST /scan/validate — implement all 6 hard-block conditions atomically. Return JSON: {status, resident_name, meal_type, block_reason}. Latency target: <2s on 4G.
Day 4	**Credit & Plan APIs**	**Harshit**	POST /admin/credit-override (manual add/deduct + reason). POST /plans/purchase (30/60/90 meal packs). GET /plans/active. POST /guest-pass/purchase.
Day 4	**Site & Meal Window APIs**	**Harshit**	GET/POST/PATCH /sites. Configure meal windows per site (Breakfast/Lunch/Dinner time ranges). Cross-site scan hard-block enforced at DB query level via site_id scoping.
Day 5	**Razorpay Integration**	**Harshit**	POST /payments/create-order, POST /payments/webhook (signature validation). Atomic credit top-up on webhook success. Failed payments: no balance change.
Day 5	**Push Notifications (FCM)**	**Harshit**	Trigger FCM on: successful scan, credit low (≤5), payment confirmed. Utility function sendNotification(resident_id, type, payload).

Phase 2 — Backend Hosting + UI Planning (Days 6–7)

Parallel work: Harshit deploys and tests backend; Rakshit designs screens and CTA flows.

PHASE 2 · DEPLOY + UI PLANNING (Days 6–7)			
Day(s)	**Task**	**Owner**	**Details / Deliverables**
Day 6	**Backend Hosting & CI**	**Harshit**	Deploy Python backend to Firebase Functions or Cloud Run. Set up staging environment. Share base API URL + Postman collection with Rakshit.
Day 6	**API Testing & Hardening**	**Harshit**	Run end-to-end tests on all 6 hard-block scenarios. Validate payment webhook with Razorpay test mode. Document all response codes.
Day 6	**UI/UX Wireframes — Resident App**	**Rakshit**	Screens: Splash/Login, Dashboard (QR + balance), Transaction History, Profile, Payment Portal, Guest Pass, Notification Center. Define all CTAs per screen.
Day 6	**UI/UX Wireframes — Vendor Scanner**	**Rakshit**	Screens: Login, Site Selection, Active Scanner (green/red result), Manual Entry Fallback. CTA: Scan, Switch Site, Manual Entry toggle.
Day 7	**UI/UX Wireframes — Super Admin**	**Rakshit**	Screens: Dashboard (live feed, stats), Resident Management (add/edit/bulk/deactivate), Site Config, Reports (weekly/monthly/financial/exception), Credit Override.
Day 7	**Screen List & Navigation Map**	**Rakshit**	Produce final screen list document with: screen name, role, key CTAs, API calls used. Share with Harshit for alignment before Day 8 frontend build start.
Day 7	**API Smoke-Test by Rakshit**	**Rakshit**	Using Postman collection from Harshit, test all endpoints to verify integration readiness before writing frontend code.

Phase 3 — Frontend Build: Web UI (PWA) + Flutter App (Days 8–11)

Both developers build in parallel. All frontend calls hit the hosted backend APIs from Day 6.

PHASE 3A · WEB PWA (iOS) · HARSHIT (Days 8–11)

Day(s)	Task	Owner	Details / Deliverables
Day 8	**PWA Project Setup**	**Harshit**	Initialise React (Vite) project. Configure manifest.json, service worker for offline QR cache. Set 'Add to Home Screen' prompt for iOS Safari.
Day 8	**Auth Screens**	**Harshit**	Login (OTP flow), role detection post-login (redirect to Resident / Vendor / Admin view). JWT token storage in memory + refresh logic.
Day 9	**Resident PWA Screens**	**Harshit**	Dashboard, QR Code display (cached offline), Balance, Transaction History (paginated), Profile, Payment Portal (Razorpay checkout embed), Guest Pass purchase.
Day 10	**Super Admin PWA Dashboard**	**Harshit**	Live scan feed (WebSocket or polling), meal counters by site, alerts panel, financial summary. Resident Management: add/edit/bulk upload/deactivate. Credit Override form.
Day 10	**Super Admin PWA — Reports**	**Harshit**	Weekly/Monthly/Financial/Exception export buttons triggering Excel download. Date-range picker for Exception Report.
Day 11	**Site & Config Management (Admin)**	**Harshit**	Add/edit PG sites, meal window time config, enable/disable meal types, assign vendor staff.
Day 11	**PWA QA & Responsive Polish**	**Harshit**	Test on iOS Safari (iPhone SE, iPhone 14). Validate offline QR display. Performance check (Lighthouse score >85). Fix layout issues.

PHASE 3B · FLUTTER APP (Android) · RAKSHIT (Days 8–11)

Day(s)	Task	Owner	Details / Deliverables
Day 8	**Flutter Project Setup**	**Rakshit**	Initialise Flutter project. Configure flavors (dev/prod). Add dependencies: dio (HTTP), firebase_core, firebase_messaging, qr_flutter, mobile_scanner, hive (local cache).
Day 8	**Auth Flow — Flutter**	**Rakshit**	Login screen with OTP via Firebase Auth. Role-based routing: Resident → ResidentNav, Vendor → ScannerNav. Persist auth token securely (flutter_secure_storage).
Day 9	**Resident Module — Flutter**	**Rakshit**	Dashboard with QR widget (offline-capable via local cache), balance chip, transaction list. Profile screen. Bottom nav: Home / History / Profile.
Day 9	**Payment & Guest Pass — Flutter**	**Rakshit**	In-app plan selection (30/60/90 meals). Launch Razorpay Flutter SDK. Guest Pass purchase flow. FCM notification listener for payment confirmation.
Day 10	**Vendor Scanner Module — Flutter**	**Rakshit**	Site selection dropdown on login. Full-screen QR scanner (mobile_scanner). Green/Red result screen

			with resident name + block reason. Auditory feedback (success chime / error buzz). Manual entry fallback.
Day 10	**Scan Result Logic & Feedback**	**Rakshit**	Parse POST /scan/validate response. Map status codes to UI states. Auto-reset scanner after 1.5s delay. Display last-scanned resident summary panel.
Day 11	**Push Notifications — Flutter**	**Rakshit**	Integrate Firebase Cloud Messaging. Handle foreground + background notifications. Show in-app banners for scan success, low credit alert, payment confirmation.
Day 11	**Flutter App QA**	**Rakshit**	Test on Android 9, 11, 13 physical/emulated devices. Validate all API integrations. Check offline QR display. Fix crashes and layout issues.

Phase 4 — UAT & Integration Testing (Days 12–13)

Joint activity. Both developers + client tester.

PHASE 4 · UAT · JOINT (Days 12–13)			
Day(s)	**Task**	**Owner**	**Details / Deliverables**
Day 12	**End-to-End Integration Test**	**Harshit**	Test full scan flow: Resident QR → Vendor scan → credit deduct → FCM push → Admin live feed update. All 6 block conditions verified.
Day 12	**Cross-Platform Test**	**Rakshit**	Run Flutter app and PWA side-by-side. Confirm identical behaviour on same API calls. Verify iOS PWA offline QR works after Add to Home Screen.
Day 12	**Stress Test — 100 Concurrent Scans**	**Harshit**	Simulate 100 simultaneous scan requests on staging. Monitor Firestore latency, atomic credit deduction integrity, no duplicate deductions.
Day 12	**Payment Flow Live Test**	**Harshit**	Switch Razorpay to production mode (test credentials). Complete real micro-transaction (₹1). Verify webhook fires and balance updates. Revert to test mode after.
Day 13	**Client Walkthrough Session**	**Harshit**	Demo all 3 modules to Manish: Resident PWA, Vendor Scanner App, Admin Dashboard. Walk through reports export, credit override, site config.
Day 13	**Bug-Fix Sprint**	**Rakshit**	Triage and fix all bugs raised during client walkthrough. Priority: Critical and High issues must be resolved before Day 14 deployment.
Day 13	**UAT Sign-Off**	**Harshit**	Obtain written UAT sign-off from client. Confirm no critical/high bugs remain. Trigger formal deployment go-ahead.

Phase 5 — Deployment & Launch (Day 14)

PHASE 5 · DEPLOYMENT (Day 14)			
Day(s)	**Task**	**Owner**	**Details / Deliverables**
Day 14	**PWA Production Deployment**	**Harshit**	Deploy React PWA to Firebase Hosting / Vercel. Configure custom domain (if required). Verify 'Add to Home Screen' on iOS Safari. Share live PWA URL with client.
Day 14	**Backend Production Deploy**	**Harshit**	Promote staging Firebase Functions to production. Enable Razorpay live keys (swap from test). Final Firestore rules review (deny unauthorized reads). Set up uptime monitoring.
Day 14	**Play Store Submission**	**Rakshit**	Build signed release APK/AAB. Complete Play Store listing (description, screenshots, privacy policy). Submit for review or publish to Internal Testing track. Share TestFlight-equivalent link.

Day 14	**Admin Credential Handover**	**Harshit**	Hand over: Super Admin login, Firebase console access, Razorpay dashboard, Vercel/Firebase Hosting dashboard. Document all credentials in secure handover sheet.
Day 14	**Training Session (2 hrs)**	**Harshit**	Live video call: Super Admin operations, vendor staff scanning guide, resident onboarding. Deliver Admin PDF Manual + Vendor Quick-Start Guide.

3. Technical Stack Reference

Layer	Technology	Owner	Notes
Backend Language	Python (FastAPI / Flask)	**Harshit**	REST API, async support
Backend Hosting	Firebase Cloud Functions / Cloud Run	**Harshit**	Auto-scaling, serverless
Database	Firebase Firestore	**Harshit**	Real-time sync, site_id scoping
Authentication	Firebase Auth (OTP + email)	**Harshit**	Mobile-first, role middleware
Push Notifications	Firebase Cloud Messaging (FCM)	**Harshit**	Android + PWA
Payment Gateway	Razorpay	**Harshit**	UPI / Card / Net Banking + webhooks
Web UI (iOS PWA)	React + Vite + Tailwind CSS	**Harshit**	Service worker, offline QR cache
PWA Hosting	Firebase Hosting / Vercel	**Harshit**	CDN-backed, low latency
Mobile App	Flutter (Dart)	**Rakshit**	Single codebase → Android APK
App Distribution	Google Play Store	**Rakshit**	Production + internal track
QR Scanning	mobile_scanner (Flutter package)	**Rakshit**	Camera-based, real-time decode
Local Cache (App)	Hive (Flutter)	**Rakshit**	Offline QR storage
HTTP Client (App)	dio (Flutter)	**Rakshit**	Interceptors, auth token refresh
Design / Wireframes	Figma (or equivalent)	**Rakshit**	Shared with Harshit before Day 8

4. API Contract Summary

All endpoints must be implemented and tested by Day 6. Rakshit will use these for both PWA review and Flutter integration.

Method	Endpoint	Role	Description
POST	/auth/login	All	OTP / email login, returns JWT
GET	/resident/profile	Resident	Profile data incl. room, PG site
GET	/resident/qr-code	Resident	Returns base64 signed QR image
GET	/resident/balance	Resident	Current credit count + plan expiry
GET	/resident/transactions	Resident	Paginated scan history
POST	/scan/validate	Vendor	Core scan engine — all 6 hard-block checks
GET	/sites	Vendor/Admin	List sites with meal window config
POST	/payments/create-order	Resident	Create Razorpay order for plan/guest pass
POST	/payments/webhook	System	Razorpay webhook — validates sig, credits balance
GET	/admin/residents	Admin	Paginated resident list with filters
POST	/admin/residents	Admin	Add individual resident
POST	/admin/residents/bulk	Admin	CSV/Excel bulk import
PATCH	/admin/residents/:id	Admin	Edit resident profile
DELETE	/admin/residents/:id	Admin	Soft-delete, invalidates QR
POST	/admin/credit-override	Admin	Manual credit add/deduct + reason log
GET	/admin/reports/weekly	Admin	Excel download — attendance report
GET	/admin/reports/monthly	Admin	Aggregated monthly summary
GET	/admin/reports/financial	Admin	Payment transaction log
GET	/admin/reports/exception	Admin	Blocked scan log by date range
POST	/guest-pass/purchase	Resident/Admin	Issue single-use guest QR

5. Inter-Developer Handoff Points

These are the critical synchronisation points where Harshit must deliver to Rakshit before work can proceed.

Day	Deliverable	From → To	Why It's Needed
Day 1	Firestore Schema Doc	Harshit → Rakshit	Rakshit needs field names to design accurate UI screens and plan Flutter models
Day 6	Hosted API URL + Postman Collection	Harshit → Rakshit	Rakshit needs live endpoints to smoke-test before writing frontend API calls
Day 6	FCM Integration Guide	Harshit → Rakshit	Flutter app needs Firebase project config (google-services.json) from Harshit's Firebase project
Day 7	Screen List & CTA Map	Rakshit → Harshit	Harshit builds Web PWA to match agreed screens; prevents rework
Day 11	Flutter APK (debug build)	Rakshit → Harshit	Harshit runs end-to-end stress test using Flutter debug APK against production API
Day 13	Signed Release APK / AAB	Rakshit → Harshit	Harshit coordinates final deployment checklist; Rakshit handles Play Store submission independently

6. Communication & Workflow Norms

- Daily sync: 15-minute WhatsApp/call check-in each morning to flag blockers.
- All code to be pushed to a shared Git repository. Branch naming: feature/harshit-* and feature/rakshit-*.
- API changes must be communicated to the other developer with at least 1 day's notice.
- Any scope creep from client must follow the formal Change Request process — do not absorb unilaterally.
- All bugs found during UAT to be logged in a shared Google Sheet: Bug ID, Severity, Owner, Status.
- Documentation (Admin Manual, Vendor Guide) to be drafted by Harshit in parallel with Phase 4.

7. Definition of Done

The project is considered complete when all of the following criteria are met:

Backend	All 20 API endpoints return correct responses. All 6 hard-block conditions pass automated tests. Payment webhook validates signature and credits atomically.
Web PWA	Loads and functions on iOS Safari 16+. QR displays offline (service worker cached). Admin dashboard renders live scan feed. All report exports produce valid Excel files.
Flutter App	Installs and runs on Android 9.0+. Scanner achieves <2s scan-to-result. FCM notifications received in foreground and background. Published to Play Store (internal track or production).
Deployment	Backend on production Firebase environment. PWA accessible via public URL. Admin credentials handed over. Razorpay live keys active.

Sign-Off	UAT checklist completed and signed by client (Manish). Zero critical or high open bugs. Training session completed.